PROJET S8

QUADRUPÈDE 1

RAPPORT TECHNIQUE

Rédigé par :

Noms	**Prénoms**
AKPO	Darling-Mariah
COUTHON	Cadoirelle
GAUTHO	Emmanuel
MICHELET	Antonin
TOMEDE	Hékey

TABLE DES MATIERES

PAGE

1. CONTEXTE & OBJECTIFS 5
2. APPROCHE AVEC REALSENSE ..... ERROR! BOOKMARK NOT DEFINED.
2.1. Traitement d'image classique : Canny+ Hough 6
2.2. Deep Learning : Yolo 9
2.3. RANSAC +PCL 11
2.4. Teachable Machine 12
3. APPROCHE AVEC LIDAR 15
4. LOCOMOTION 25
5. CONCLUSION 33
BIBLIOGRAPHIE 34
ANNEXES 35

Liste des figures

FIGURE 2.1. TITRE ..... ERROR! BOOKMARK NOT DEFINED.

Liste des tableaux

AUCUNE ENTREE DE TABLE D'ILLUSTRATION N'A ETE TROUVEE.

1. CONTEXTE & OBJECTIFS

Dans le cadre du projet S8, nous sommes amenés à concevoir et étudier des solutions permettant à un robot quadrupède de naviguer de manière autonome dans un environnement comportant des escaliers.

Cette problématique s'inscrit dans le domaine de la robotique mobile, où l'évolution en environnement complexe constitue un enjeu majeur. Les escaliers représentent un défi particulier, nécessitant une perception fiable, une estimation de leur géométrie et une adaptation de la locomotion.

Ainsi, ce projet aborde la détection des escaliers, l'analyse de leurs caractéristiques et le contrôle du déplacement du robot afin d'assurer une navigation autonome et robuste.

Les objectifs principaux de ce projet sont donc :

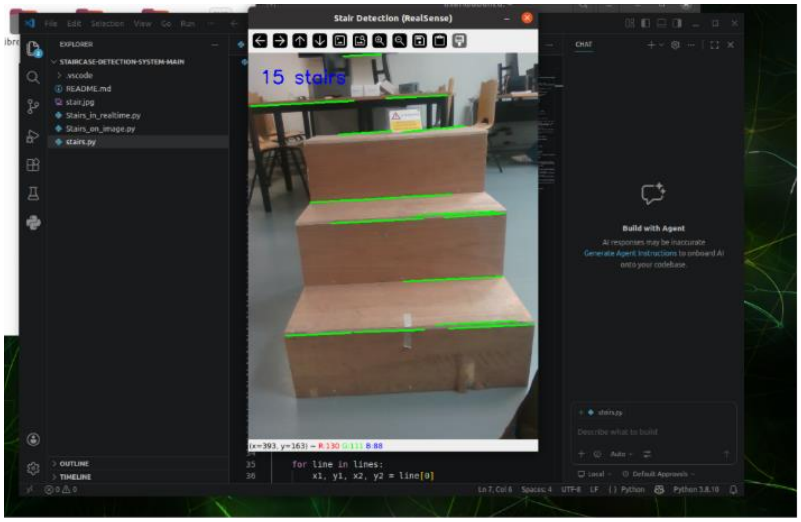
- Réaliser un état de l'art pour une meilleure compréhension des outils et méthodes à utiliser ;
- Étudier les techniques de perception et de détection des escaliers ;
- Extraire les caractéristiques géométriques des escaliers ;
- Adapter la locomotion du robot en fonction des contraintes identifiées.

2. APPROCHE AVEC REALSENSE

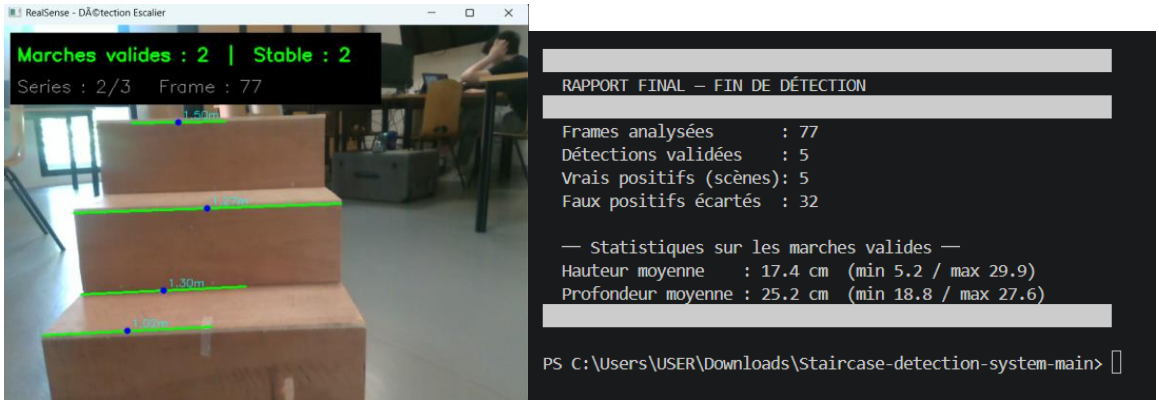
2.1. TRAITEMENT D'IMAGE CLASSIQUE : CANNY+ HOUGH

Liens : **https://github.com/rushi-gif/Staircase-detection-system**

Ce projet est une application Python légère de détection d'escalier en temps réel, destinée principalement aux personnes malvoyantes. Il utilise la bibliothèque OpenCV pour analyser le flux vidéo RGB de la caméra et détecter les lignes horizontales caractéristiques des marches via des algorithmes classiques de traitement d'image (Canny, transformée de Hough). Lorsqu'un escalier est détecté, le système génère une alerte audio pour avertir l'utilisateur. Cette approche n'exploite pas l'information de profondeur de la D435i et reste donc sensible aux variations d'éclairage et aux faux positifs sur des textures similaires à des escaliers.



Après modifications de ce dépôt, on obtient les résultats suivants :



Les résultats présentés sont issus d'un système de détection développé en Python, exploitant directement le SDK RealSense sans ROS. Le pipeline repose sur une

détection de contours (Canny), une transformée de Hough pour extraire les lignes horizontales, et une reconstruction géométrique 3D à partir des intrinsèques de la caméra pour estimer hauteur et profondeur de chaque marche.

Performances globales

Sur **77 frames analysées**, le système a produit **5 détections validées** avec **32 faux positifs écartés** par le mécanisme de cohérence temporelle. Le taux de rejet est efficace, démontrant la robustesse du filtrage par fenêtre glissante implémenté dans le code.

Analyse de la hauteur

La hauteur moyenne mesurée est de **17,4 cm** pour une cible de **19 cm**, soit une erreur absolue de **1,6 cm (8,4%)**. La distribution révèle deux clusters distincts sans aucune mesure centrée sur la valeur cible :

- Les marches de type 3 sont systématiquement estimées entre **5 et 8 cm**, probablement dues à une ligne de Hough détectée sur le nez de marche plutôt que sur la contremarche
- Les marches de type 1 sont estimées entre **27 et 30 cm**, légèrement surestimées

Cette bimodalité suggère que la reconstruction géométrique par différence d'axe Y souffre d'une imprécision sur l'association entre les lignes détectées et les vraies arêtes des marches.

Analyse de la profondeur

La profondeur moyenne est de **25,2 cm** pour une cible de **25 cm**, soit une erreur de seulement **0,2 cm (0,8%)**. C'est le paramètre le mieux estimé du système, ce qui est cohérent avec la précision de la mesure de profondeur de la D435i sur l'axe Z.

Stabilité inter-frames

Les mesures sont remarquablement stables sur les frames consécutives 71 à 77 :

La variance reste inférieure à 2 cm sur l'ensemble des frames, confirmant la cohérence temporelle du système.

Limites observées

La marche 2 est systématiquement rejetée comme "hors norme" dans toutes les détections, avec une profondeur anormalement faible (1,9 à 4,4 cm). Cela indique qu'une ligne parasite est régulièrement détectée par Hough entre deux vraies arêtes, vraisemblablement due à une texture ou un reflet sur la surface des marches.

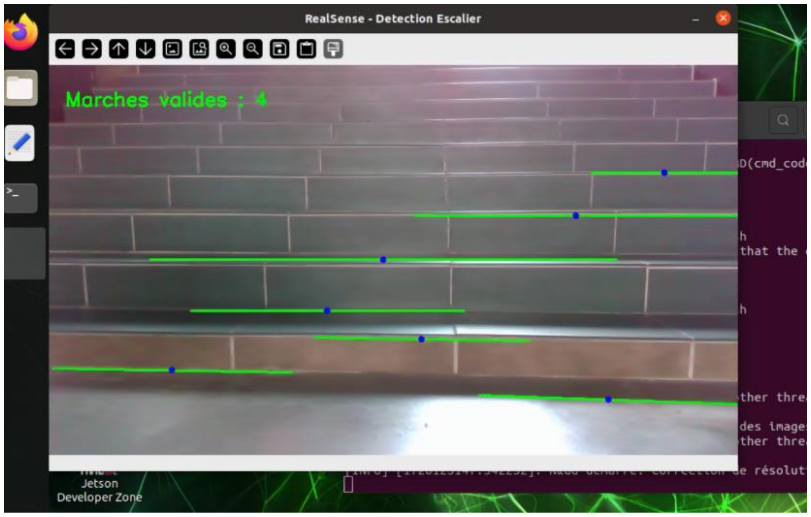
Conclusion

Le système démontre une excellente précision sur la **profondeur** (< 1% d'erreur) et une bonne **stabilité temporelle**. La principale limitation porte sur l'estimation de la hauteur absolue, biaisée par la nature de la détection de lignes qui ne garantit pas l'association systématique avec les vraies arêtes géométriques des marches. Une amélioration possible serait d'intégrer une contrainte de continuité géométrique entre frames successives pour stabiliser l'association ligne/marche.

Paramètre	Frame 71	Frame 75	Frame 77	Écart max
Hauteur marche 1	29,9 cm	29,4 cm	28,3 cm	1,6 cm
Profondeur marche 1	25,0 cm	24,7 cm	23,1 cm	1,9 cm
Hauteur marche 3	6,6 cm	5,2 cm	5,5 cm	1,4 cm

- **Détection avec realsense du robot :**

Terminaux	Commandes
1	cd Downloads source /opt/ros/noetic/setup.bash sudo start systemctl realsence pyhton3 test3_camera_metrics
2	rostopic echo /escalier_detecte



2.2. DEEP LEARNING : YOLO

YOLO signifie You Only Look Once. C'est un algorithme de détection d'objets en temps réel.

La détection d'escaliers avec YOLO repose sur une chaîne de traitement complète allant de la constitution d'un dataset jusqu'à l'obtention d'une perception exploitable pour la robotique. Cette section décrit de manière détaillée l'ensemble du processus, afin de comprendre comment un modèle de détection est construit, entraîné, évalué et utilisé pour la perception.

1. Constitution du dataset et annotation

La première étape consiste à fournir au modèle des exemples représentatifs de l'objet à détecter. Pour cela, un dataset a été constitué en capturant différents types d'escalier sous différents angles, distances et conditions d'éclairage. L'objectif est de couvrir un maximum de variations visuelles afin que le modèle apprenne une représentation robuste de l'objet.

Chaque image a ensuite été annotée manuellement. L'annotation consiste à tracer une bounding box autour de l'escalier et à lui associer une classe, ici « Stairs ». Ces annotations servent de vérité terrain (ground truth) pendant l'entraînement : elles indiquent au modèle ce qu'il doit apprendre à reconnaître et où se trouve l'objet dans l'image.

2. Entraînement du modèle

L'entraînement consiste à présenter au modèle les images annotées afin qu'il apprenne à prédire correctement la position et la classe de l'escalier. Pour chaque image, YOLO génère une prédiction initiale (souvent incorrecte au

début). Cette prédiction est comparée à l'annotation réelle, ce qui permet de calculer une erreur appelée loss. Le modèle ajuste alors ses paramètres internes pour réduire cette erreur.

Ce processus est répété sur plusieurs epochs. Au fil des itérations, le modèle améliore progressivement sa capacité à localiser l'escalier et à reconnaître sa classe. Les métriques telles que la loss, la précision, le recall et le mAP permettent de suivre l'évolution de l'apprentissage.

3.Test du modèle et interprétation des résultats

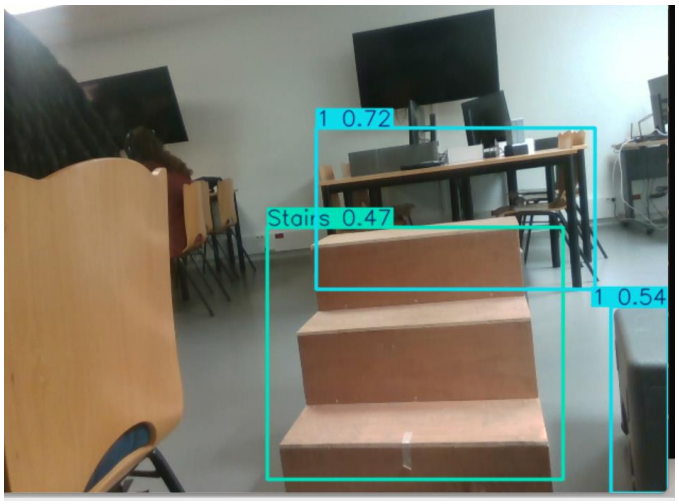
Une fois l'entraînement terminé, le modèle est testé. YOLO renvoie pour chaque détection trois informations essentielles : une bounding box, une classe et un score de confiance.

La bounding box correspond au rectangle englobant l'objet détecté. La classe indique ce que le modèle pense avoir reconnu (ici « Stairs »). Le score de confiance représente la probabilité estimée par le modèle que la détection soit correcte.

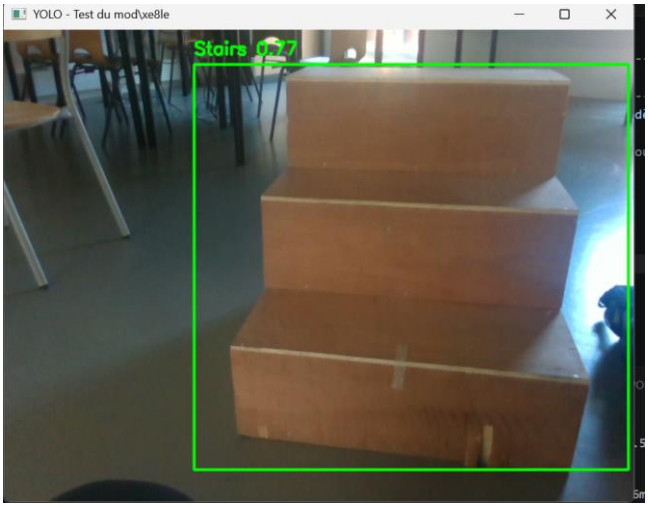
Dans les tests réalisés, les résultats affichent par exemple « Stairs 0.47 » ou « Stairs 0.77 ». Cela signifie que YOLO a détecté un escalier avec une confiance de 47 % ou 77 %. Les scores faibles observés au début sont normaux : ils reflètent un modèle qui a besoin d'être réentraîné pour améliorer sa précision.

Résultats obtenus :

1er entraînement :



2ème entraînement :



Après réentraînement du modèle YOLOv8, les courbes de pertes (voir annexe) montrent une diminution plus nette, tandis que les métriques de performance (precision, recall, mAP50, mAP50-95) augmentent de manière significative. Ces évolutions indiquent un apprentissage plus efficace et une meilleure généralisation. Les tests sur images réelles confirment cette amélioration : la confiance de détection passe d'environ 0.47 à 0.77 . Le modèle détecte désormais les escaliers de manière plus stable et plus fiable

4.Passage de la détection 2D à la perception 3D

YOLO fournit uniquement une information 2D : la position de l'escalier dans l'image. Pour obtenir une perception exploitable par un robot, il est nécessaire d'extraire des informations 3D. Cela se fait en fusionnant la détection YOLO avec les données de profondeur fournies par une caméra RealSense.

La caméra fournit pour chaque pixel une valeur de profondeur. En récupérant les pixels situés à l'intérieur de la bounding box détectée par YOLO, il est possible d'extraire la géométrie de l'escalier : hauteur de marche, giron, largeur, inclinaison et position dans l'espace. Cette fusion permet de transformer une simple détection visuelle en une perception tridimensionnelle utilisable pour la locomotion d'un robot quadrupède.

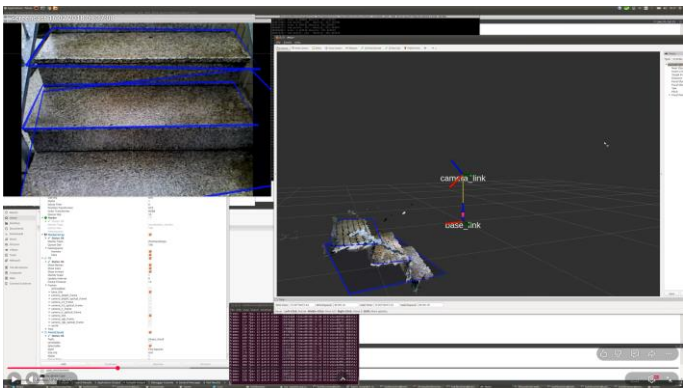
2.3. RANSAC +PCL

Ce projet est un nœud ROS écrit en C++ qui détecte les escaliers à partir d'un nuage de points 3D. Il utilise l'algorithme RANSAC via la bibliothèque PCL pour

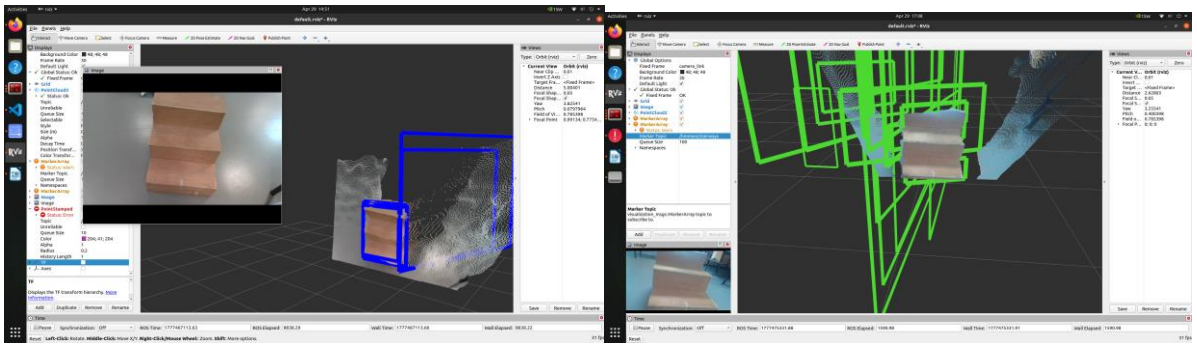
segmenter les plans géométriques présents dans la scène. Les plans horizontaux parallèles et régulièrement espacés sont identifiés comme les marches de l'escalier. Le système fournit en sortie les paramètres géométriques de l'escalier détecté, tels que la hauteur, la profondeur et l'angle des marches. Cette approche exploite pleinement l'information de profondeur de la D435i mais nécessite une intégration ROS et présente des limitations en termes de performance temps réel sur des nuages de points denses.*

Liens : **GitHub - ne0h/ros_stairsdetection: ROS node to detect (find) stairs based on RANSAC and PCL. · GitHub**

Résultats attendus :



Résultats obtenus :



2.4. TEACHABLE MACHINE

Teachable Machine est une plateforme web développée par Google permettant de créer et tester rapidement des modèles d'apprentissage automatique sans écrire de code. Elle propose trois types de modèles : *image*, *audio* et *pose*.

Le but ici est d'entraîner un modèle de classification d'images capable de reconnaître un escalier et d'en extraire des métriques géométriques. Le flux complet inclut : collecte d'images (création de deux classes mur et escaliers), entraînement via *Teachable Machine*, export du modèle, intégration dans un script Python, et tests sur PC.
Pour entraîner le modèle, une base d'images d'escaliers a été constituée :

- Photos prises dans différents environnements (hall, salle de classe, couloirs…)
- Variations d'angles : face, côté, plongée, contre-plongée
- Variations de luminosité
- Objectif : garantir une bonne robustesse du modèle face aux conditions réelles du robot.

Interprétation des valeurs affichées

"0 Stairs (1.00)" Le modèle Teachable Machine (exporté en TensorFlow Lite) renvoie :

- 0 = Classe Escalier
- Score 1.00 = 100 % de confiance

"EXTRACTION OK" Indique que toutes les étapes du script Python ont fonctionné :

- Chargement du modèle
- Détection des bords (Canny)
- Extraction des lignes (Hough)
- Calcul des métriques
- Aucun échec dans la géométrie détectée

Inclinaison : 51.9°

Angle calculé à partir de la ligne principale détectée sur l'escalier.

$$\theta = \arctan(\Delta y \backslash \Delta x)$$

Largeur : 0.98 m

Mesure horizontale entre les deux bords latéraux détectés : Distance en pixels → conversion en mètres via **matrice de calibration**

Hauteur : 0.21 m

Hauteur d'une marche (contremarche)

Profondeur : 0.17 m

Profondeur (giron) d'une marche : Mesure horizontale entre deux nez de marche

Résultat obtenu :



3. APPROCHE AVEC LIDAR

3.1. DANS LE CADRE DU PROJET

Le principal avantage du LiDAR est sa robustesse et sa moindre sensibilité à la lumière mais il y a certains désavantages comme le cout de calcul, les points morts et la qualité des nuages de points.

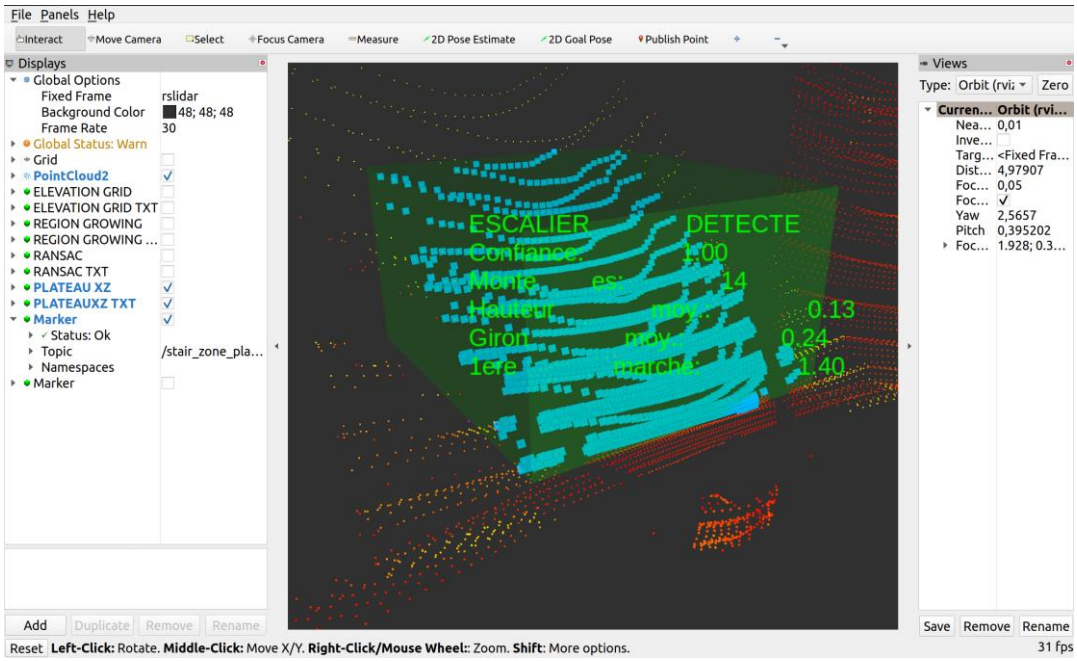
Pour détecter les escaliers une méthode d'apprentissage semble plus robuste mais plus difficile à mettre en place. Cependant pour l'extraction de la géométrie elle sera surement indispensable et beaucoup plus efficace.

Le plus simple à mettre en place pour le moment et pour un cas simple semble être la segmentation de plans suivi des contraintes géométriques, par la suite des méthodes pourront être combinés entre elles pour avoir de meilleurs résultats, s'il y a des résultats dans un 1er temps.

L'exploitation du nuage de points étant assez lourde en calcul, également le nuage de point contient beaucoup d'informations. Il est donc nécessaire de limiter la zone d'exploitation. Il faut alors en amont développer un nœud ROS2 qui permettra de filtrer les points ; on définit une ROI (Region Of Interest) qui délimite une zone devant le robot. Puis seulement les points contenus dans cette zone seront envoyés aux algorithmes que l'on testera, pour limiter le nombre de calculs.

3.2. ESSAIE DES DIFFERENTS ALGORITHMES

Dans le nuage de points PointsCloud2 affiché dans RViz2 une boite transparente verte apparait. C'est là qu'est la zone probable où se situe l'escalier. Les points appartenant à l'escalier apparaissent en orange gras. Pour l'instant seulement la rupture de hauteur qui est utilisé.

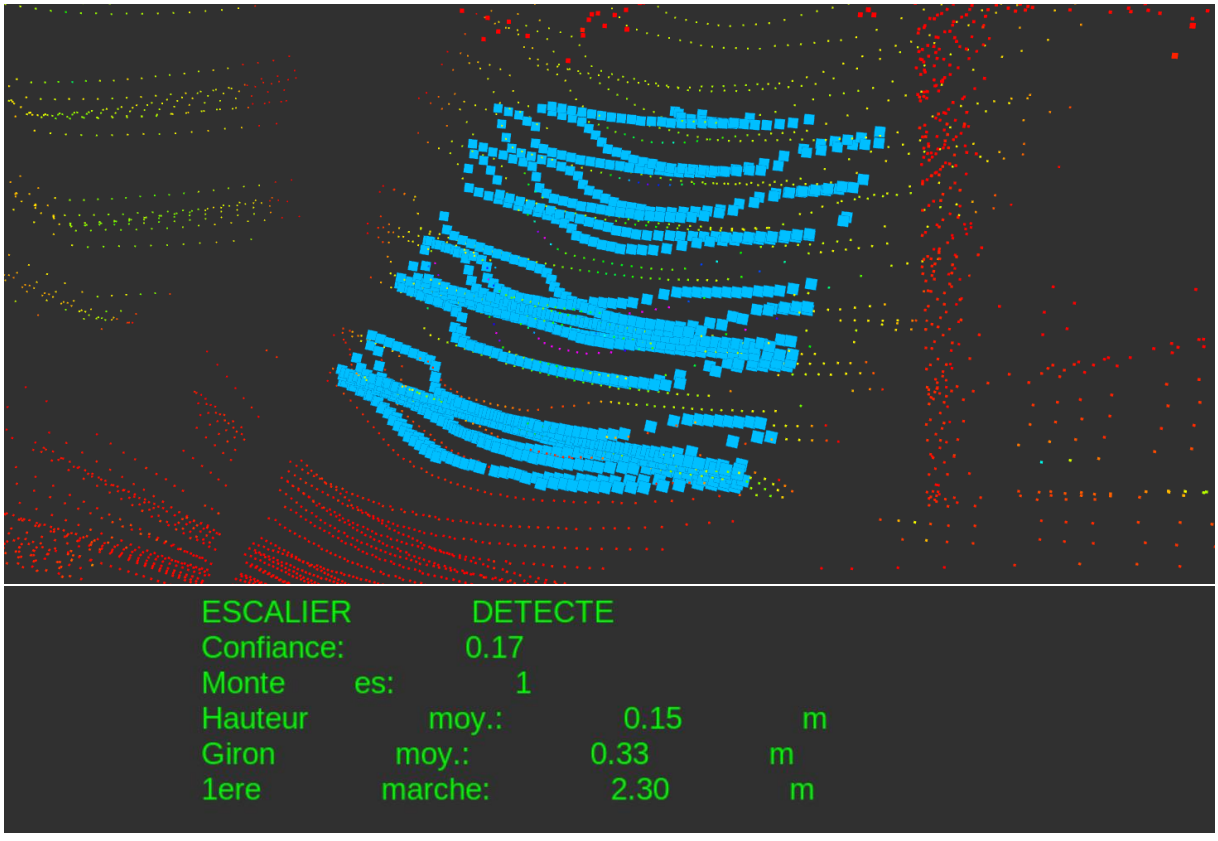


Les tests ont été réalisés à partir d'un rosbag enregistré avec le robot immobile face à l'escalier du bâtiment Turing. Cette configuration permet de comparer les algorithmes sur un même nuage de points et d'éviter que les différences de résultats soient liées au déplacement du robot. Les dimensions réelles de l'escalier ont été mesurées manuellement afin de disposer d'une référence : une hauteur de marche de 16 cm et un giron de 28,5 cm. Les algorithmes ont ensuite été exécutés sur le même échantillon, avec deux tailles de ROI différentes, afin d'évaluer leur sensibilité à la zone d'analyse. Le donne des mesures toutes les 0,1 secondes, la mesure affichée est un lissage des 50 dernières mesures.

3.2.1. RANSAC

RANSAC (RANdom SAmple Consensus) permet de détecter des plans dans un nuage de points, même lorsque les données sont bruitées. L'algorithme teste plusieurs plans possibles à partir de points choisis aléatoirement, puis conserve celui qui regroupe le plus grand nombre de points cohérents. Dans notre cas, il peut servir à retrouver les surfaces planes de l'escalier, comme les marches, les contremarches et le sol. Son avantage est d'être robuste au bruit, mais ses résultats dépendent beaucoup des paramètres choisis.

Il y a plusieurs paramètres a réglé comme la forme géométrique a détecté, le seuil de distance, le nombre d'itérations, le nombre minimal de points, le nombre de points tirés qui nous donne un score en relation avec le nombre de points cohérents avec le plan détecté.

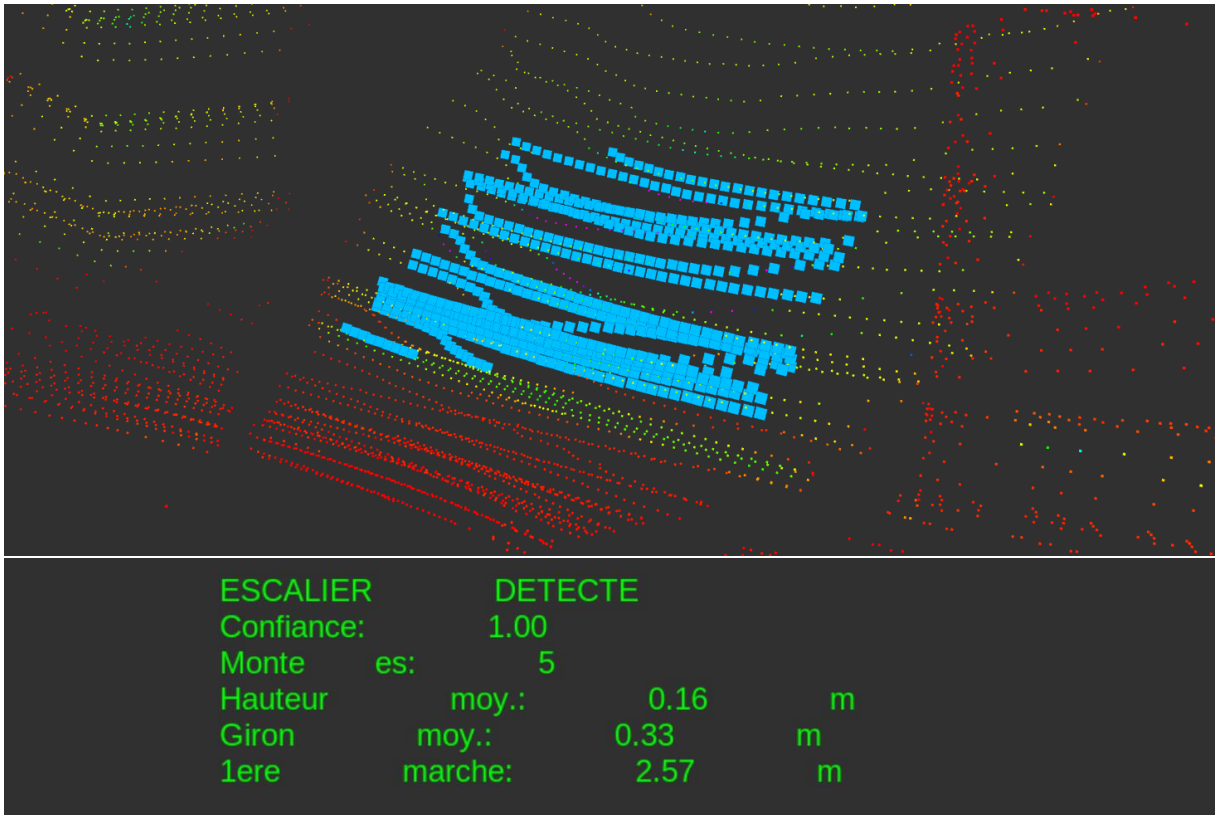


Les résultats obtenus sont assez proches de la réalité mais il y a tout de même une forme surestimation du giron.

Il est possible que cette surestimation soit dû au fait que le sol n'était pas filtré dans un premier temps. Il a plus tard été filtré.

3.2.2. PLATEAU XZ

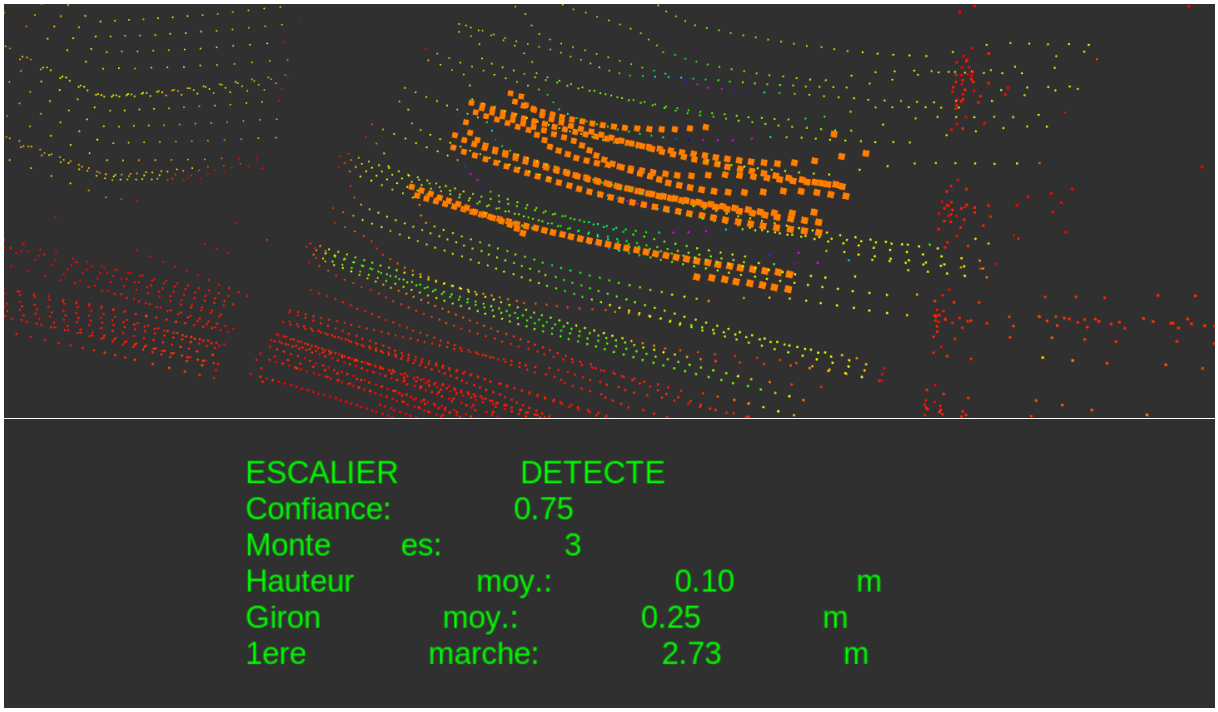
La méthode Plateau XZ consiste à projeter le nuage de points LiDAR dans un plan vertical XZ afin d'analyser le profil de l'escalier. Elle permet de repérer plus simplement les ruptures de hauteur correspondant aux marches, puis d'estimer le giron, la hauteur moyenne et la distance de la première marche.



La hauteur semble exacte mais le giron est lui sur estimé.

3.2.3. ELEVATION GRID

La méthode Elevation Grid consiste à transformer le nuage de points LiDAR en une grille 2D où chaque cellule contient une information de hauteur. Cette représentation permet d'identifier les variations verticales du terrain et de repérer les ruptures de hauteur pouvant correspondre aux marches d'un escalier



La hauteur est largement sous-estimée et le giron est légèrement sous-estimé.

3.2.4. CHOIX POUR L'IMPLEMENTATION

A ce point on choisit de garder les plus performants et les plus réalistes. Je choisi donc de garder plateau xz et ransac. Dans la partie suivante je vais comparer les 2 algorithmes avec des paramètres de détections différents.

Pour comparer les différentes méthodes, plusieurs critères ont été retenus : la précision de l'estimation du giron, la précision de l'estimation de la hauteur de marche, la stabilité des mesures dans le temps, la sensibilité à la taille de la ROI, ainsi que la faisabilité d'une implémentation en temps réel sur l'ordinateur embarqué du robot. Ces critères permettent de ne pas seulement évaluer la justesse des résultats, mais aussi la robustesse de chaque méthode dans un contexte robotique réel.

Pour pouvoir les comparer on a généré un fichier .json qui enregistre toutes les mesures du giron et de la hauteur en fonction du temps avec le robot immobile devant les escaliers. J'ai ensuite extrait les moyennes et médianes pour voir lesquelles se rapprochaient le plus de la réalité, sachant que dans les faits la hauteur de marche est de 16 centimètres et le giron de 28.5 centimètres, pour les escaliers du bâtiment Turing.

Les résultats apparaissent dans le tableau suivant :

	ROI	Giron (en cm)			Hauteur (en cm)		
		Médiane	Moyenne	Erreur	Médiane	Moyenne	Erreur
Ransac	Grande	17.11	17.06	40,1%	18.12	18.06	12,9%
	Petite	33.18	32.97	15,7%	19.25	19.12	19,5%
Plateau XZ	Grande	29.68	29.55	3,7%	17.33	17.47	9,2%
	Petite	23.12	22.41	21,4%	12.24	13.02	18,6%

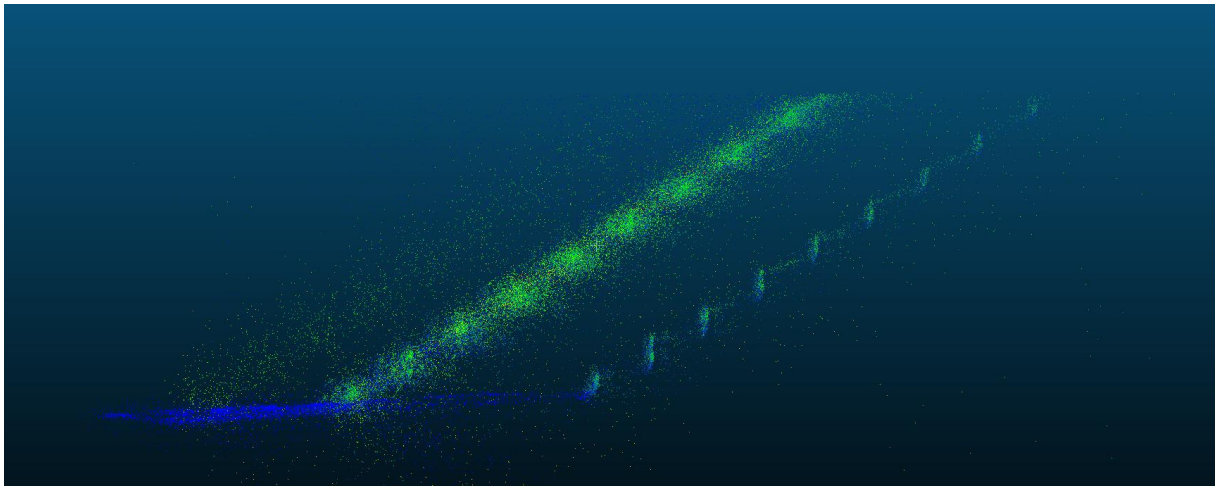
On voit donc que le Plateau XZ est plus précis avec une petite ROI tandis que le Ransac est plus performant avec une grande ROI. Cependant les résultats les plus proches de la réalité sont ceux du "plateau XZ" avec une ROI grande.

C'est donc la méthode qui semble la plus pertinente à implémenter dans l'ordinateur embraqué du robot.

Ce sera donc le plateau XZ que l'on implémentera dans la solution finale.

3.2.5. LIMITES

Les limites auxquelles j'ai été confronté sont surtout lié aux points aberrants. Sur certaines surfaces le faisceau laser renvoyer au LiDAR n'est pas correct, ainsi la distance peut être surestimé ou sous-estimé. En l'occurrence la surface des escaliers qui est un carrelage qui reflète un peu la lumière donne quelques points faux. On le voit sur l'image suivant, une section vue de côté des escaliers. On voit en effet qu'il semble y avoir plusieurs "couches" d'escaliers. Ces couches peuvent alors fausser les résultats, en réduisant ou agrandissant la zone de plateau détecté ou faussant la distance entre les points.



Pour palier a ces limites il faut alors il faut alors filtrer les points avec précautions pour éliminer les faux positifs.

Une autre limite est le nombre de points que le robot reçoit en temps réel. Plus il y a de points plus c'est possible d'être précis mais plus le LiDAR est performant plus le cout du capteur augmente mais la quantité de calcul augmente également avec le nombre de points.

Il y a également le fait que le lidar soit au bas de l'escalier. Il y a donc certaines zones aveugles ou on ne peut pas détecter la vraie géométrie de l'escalier, il détecte le plan de la marche seulement pour les 2 premières marches.

4. APPROCHE MULTI-CAPTEURS

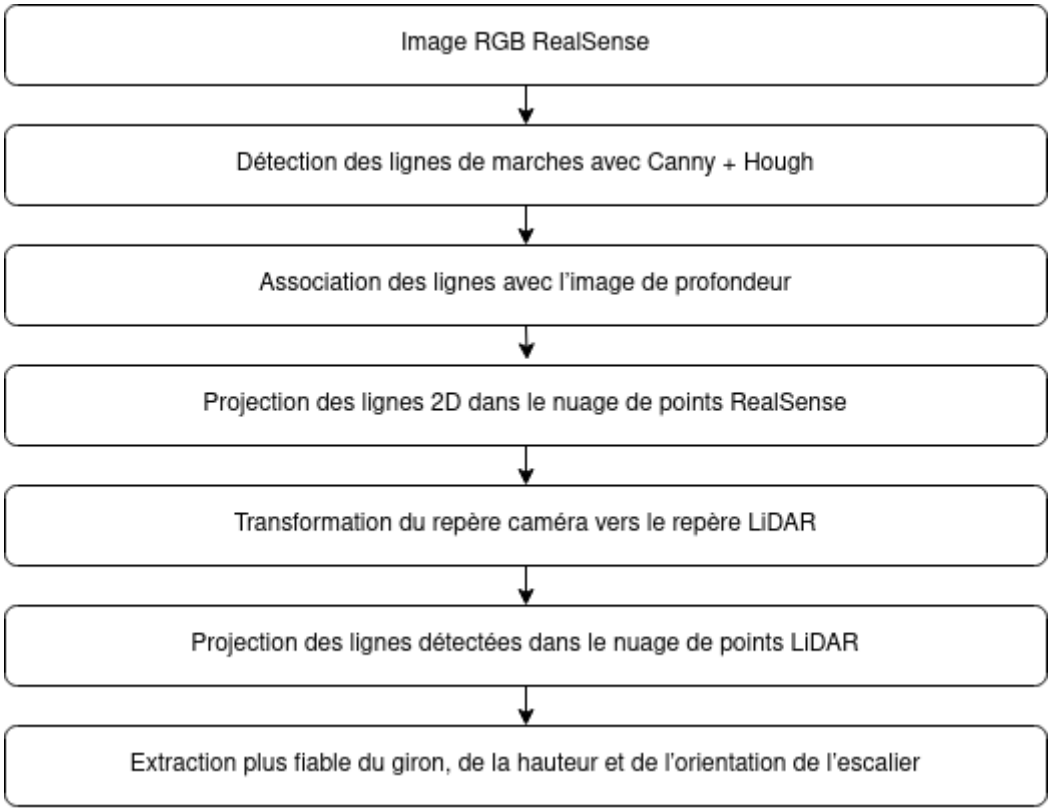
Une approche multi-capteurs a également été envisagée afin de combiner les avantages de la caméra RealSense et du LiDAR. L'objectif était d'utiliser la caméra pour détecter visuellement les lignes caractéristiques de l'escalier, puis d'exploiter les informations de profondeur et le nuage de points LiDAR pour obtenir une estimation plus fiable de la géométrie.

La caméra RealSense présente l'avantage de fournir une image RGB ainsi qu'une information de profondeur dense sur la zone proche du robot. Elle est donc intéressante pour détecter les arêtes visibles des marches à l'aide de méthodes de traitement d'image comme Canny et Hough. De son côté, le LiDAR fournit un nuage de points 3D plus robuste aux variations de lumière et permet d'obtenir une représentation géométrique de l'environnement.

Le principe envisagé était le suivant : dans un premier temps, les lignes correspondant aux marches auraient été détectées sur l'image de la caméra. Ces lignes 2D auraient ensuite été projetées dans le nuage de points de la caméra RealSense grâce à l'image de profondeur et aux paramètres intrinsèques de la caméra. Une fois ces lignes converties en informations 3D dans le repère caméra, elles auraient pu être projetées dans le repère du LiDAR à condition d'avoir calibré les deux capteurs entre eux.

Techniquement, il ne s'agit pas réellement de calibrer les nuages de points eux-mêmes, mais plutôt de calibrer les repères des deux capteurs. Il faut connaître la transformation géométrique entre le repère de la caméra et le repère du LiDAR, c'est-à-dire la rotation et la translation permettant de passer d'un repère à l'autre. Cette transformation aurait permis d'aligner le nuage de points de la RealSense avec celui du LiDAR, puis de projeter les arêtes détectées par la caméra directement dans le nuage LiDAR.

Le pipeline théorique était donc le suivant :



Cette approche aurait permis de combiner la précision visuelle de la caméra avec la robustesse géométrique du LiDAR. Les lignes détectées par la caméra auraient pu servir à mieux localiser les arêtes des marches, tandis que le LiDAR aurait permis de valider leur position dans l'espace 3D et de limiter certains faux positifs liés à l'image seule.

Cependant, cette approche n'a pas pu être menée jusqu'au bout lors des tests. La principale difficulté rencontrée a été l'impossibilité d'obtenir simultanément les flux exploitables de la caméra et du LiDAR dans les conditions de test. Sans accès fiable aux deux flux en même temps, il n'était pas possible de synchroniser les données, d'aligner les nuages de points ni de valider expérimentalement la projection entre les deux capteurs.

Cette piste reste néanmoins pertinente pour la suite du projet. Pour la mettre en œuvre, il faudrait d'abord assurer l'acquisition simultanée des données caméra et LiDAR, puis réaliser une calibration extrinsèque entre les deux capteurs. Il faudrait ensuite utiliser les transformations ROS, notamment via tf ou tf_static, pour exprimer les données caméra et LiDAR dans un repère commun. Une fois cette étape réalisée, les lignes détectées sur l'image caméra pourraient être projetées dans l'espace 3D et utilisées pour améliorer l'extraction de la géométrie de l'escalier.

En conclusion, la fusion multi-capteurs n'a pas pu être validée expérimentalement dans le cadre de ce projet, mais elle constitue une amélioration intéressante. Elle permettrait potentiellement d'obtenir une détection plus robuste que l'utilisation d'un seul capteur, en combinant les informations visuelles de la RealSense avec les informations géométriques du LiDAR.

5. LOCOMOTION

La locomotion du robot quadrupède Lite3 a été développée à l'aide de l'apprentissage par renforcement avec le framework IsaacLab et l'algorithme PPO fourni par la bibliothèque RSL-R

Plateforme robotique	Deep Robotics Lite3
Framework de simulation	NVIDIA Isaac Sim + IsaacLab
Algorithme RL	PPO via RSL-RL
Objectif	Montée autonome d'escaliers
Dépôt de référence	DeepRoboticsLab/rl_training (GitHub)

L'apprentissage par renforcement présente plusieurs avantages

- La politique apprise est robuste aux perturbations grâce à la randomisation de domaine
- Deep Robotics Lab fournit un dépôt rl_training prêt à l'emploi avec les modèles USD du Lite3
- Le framework IsaacLab permet de paralléliser des milliers d'environnements sur GPU, accélérant drastiquement l'entraînement

5.1. INSTALLATION DE L'ENVIRONNEMENT DE SIMULATION

Pour cette partie se baser sur les instructions fournies sur la page officielle de Nvidia **Installation — Isaac Sim Documentation**

5.1.1. ÉTAPE 1 — CREER UN ENVIRONNEMENT VIRTUEL CONDA

Il est fortement recommandé d'utiliser Miniconda pour isoler les dépendances du projet. La version Python doit correspondre exactement à celle requise par Isaac Sim.

```
# Installer Miniconda si ce n'est pas encore fait
wget https://repo.anaconda.com/miniconda/Miniconda3-latest-Linux-x86_64.sh
bash Miniconda3-latest-Linux-x86_64.sh
```

```
# Créer l'environnement conda avec Python 3.10 (pour Isaac Sim 4.x)
conda create -n env_isaaclab python=3.10
conda activate env_isaaclab
```

5.1.2. ÉTAPE 2 — INSTALLER PYTORCH COMPATIBLE CUDA

Il faut installer une version de PyTorch avec support CUDA avant Isaac Sim. Cette étape est critique car Isaac Sim impose des contraintes de version strictes.

```
# Vérifier la version CUDA disponible sur le système
nvcc --version
```

```
# Installer PyTorch 2.5.1 avec CUDA 12.4 (adapter selon votre CUDA)
pip install torch==2.5.1 torchvision --index-url
https://download.pytorch.org/whl/cu124
```

5.1.3. ÉTAPE 3 — INSTALLER ISAAC SIM VIA PIP

Isaac Sim est disponible sur le registre PyPI de NVIDIA. Il existe deux options : installation complète ou minimale. Pour notre projet, la version minimale suffit.

```
# Installation minimale d'Isaac Sim (recommandée pour Isaac Lab uniquement)
pip install isaacsim-rl isaacsim-replicator isaacsim-extscache-physics \
    isaacsim-extscache-kit-sdk isaacsim-extscache-kit isaacsim-app \
    --extra-index-url https://pypi.nvidia.com
```

5.2. INSTALLATION D'ISAACLAB

5.2.1. ÉTAPE 4 — CLONER ISAAC LAB DEPUIS GITHUB

```
git clone https://github.com/isaac-sim/IsaacLab.git
cd IsaacLab
```

5.2.2. ÉTAPE 5 — INSTALLER ISAAC LAB

Le script d'installation installe toutes les extensions d'Isaac Lab en mode éditable (--editable), ce qui permet de modifier le code source sans réinstaller.

```
# Sur Linux
./isaaclab.sh --install
```

```
# Alternative : installation manuelle des extensions
pip install -e source/isaaclab
pip install -e source/isaaclab_assets
pip install -e source/isaaclab_rl
```

5.2.3. ÉTAPE 6 — VERIFIER L'INSTALLATION

```
# Lancer un test basique pour vérifier que Isaac Sim fonctionne
python -c "import isaacsim; isaacsim.SimulationApp()"
```

5.3. INSTALLATION DU DEPOT RL_TRAINING DE DEEP ROBOTICS

5.3.1. ÉTAPE 7 — CLONER LE DEPOT RL_TRAINING

Le dépôt rl_training doit être cloné en dehors du répertoire IsaacLab. Il contient les sous-modules Git nécessaires (modèles du robot, configurations)

```
# Cloner EN DEHORS du répertoire IsaacLab !
cd ~ # ou dans votre répertoire de projets
git clone --recurse-submodules
https://github.com/DeepRoboticsLab/rl_training.git
cd rl_training
```

5.3.2. ÉTAPE 8 — INSTALLER L'EXTENSION RL_TRAINING

L'extension doit être installée avec l'interpréteur Python qui à Isaac Lab d'installé, c'est-à-dire depuis l'environnement conda env_isaaclab.

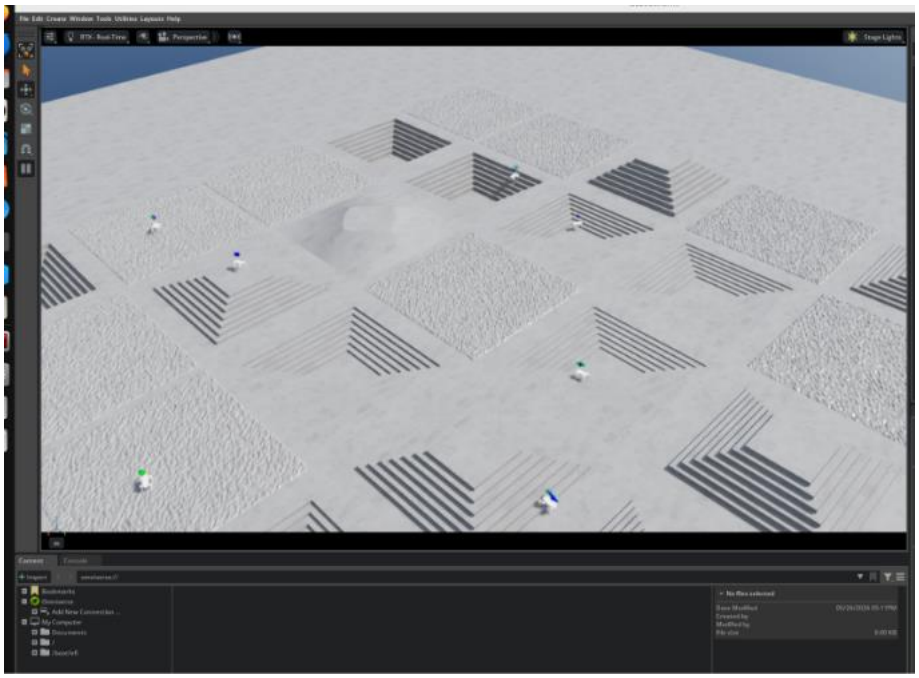
```
conda activate env_isaaclab
pip install -e source/rl_training
```

5.3.3. ÉTAPE 9 — VERIFIER LES ENVIRONNEMENTS DISPONIBLES

```
# Lister tous les environnements disponibles dans l'extension
python scripts/list_envs.py
```

Parmi les environnements listés, vous devriez choisir :

- Rough-Deeprobotics-Lite3-v0 — terrain accidenté pour le Lite3



5.4. EXPLICATION DU CODE MODIFIE

Pour ce projet le fichier du dépôt qui a été modifier est le suivant : rough_env_cfg.py je les renomer en stair_env_cfg

```python
@configclass
class DeeproboticsLite3StairEnvCfg(DeeproboticsLite3RoughEnvCfg):
    def __post_init__(self):
        super().__post_init__()

        # ---------------------------- Terrains : majorité escaliers ----------------------------
        self.scene.terrain.terrain_generator.sub_terrains["random_rough"].proportion = 0.25
        self.scene.terrain.terrain_generator.sub_terrains["hf_pyramid_slope"].proportion = 0.05
        self.scene.terrain.terrain_generator.sub_terrains["hf_pyramid_slope_inv"].proportion = 0.00
        self.scene.terrain.terrain_generator.sub_terrains["boxes"].proportion = 0.0
        self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs"].proportion = 0.45
        self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs_inv"].proportion = 0.25

        # Tu enlèves le None et tu configures la grille


        # ---------------------------- Rewards ----------------------------
        self.rewards.lin_vel_z_l2.weight = -2.0         # corps peut monter
        self.rewards.ang_vel_xy_l2.weight = -0.5   # corps peut s'incliner a gauche ou droite
        self.rewards.flat_orientation_l2.weight = -5.0 # dos peut s'incliner
        self.rewards.feet_height.params["target_height"] = 0.16 # marches de 15.5cm
        self.rewards.feet_air_time.params["threshold"] = 0.50 ## c'est le temps  qu'il faut pour qu'une pattes pose  ses pattes
        self.rewards.feet_gait.weight = 1.6              # forcer les pattes diagonal a se lever en meme temps
        self.rewards.joint_mirror.weight = -0.05     # symétrie des pattes
        self.rewards.feet_air_time_variance.weight = -5.0  # rythme libre
        self.rewards.base_height_l2.params["target_height"] = 0.35
        self.rewards.feet_slide.weight = -0.2 # pour que le robot ne glisse pas
        self.rewards.base_height_l2.weight = -14.0
```

```python
        self.rewards.feet_gait.weight = 1.6              # forcer les pattes diagonal a se lever en meme temps
        self.rewards.joint_mirror.weight = -0.05     # symétrie des pattes
        self.rewards.feet_air_time_variance.weight = -5.0  # rythme libre
        self.rewards.base_height_l2.params["target_height"] = 0.35
        self.rewards.feet_slide.weight = -0.2 # pour que le robot ne glisse pas
        self.rewards.base_height_l2.weight = -14.0


        self.rewards.feet_stumble.weight = -0.4  #  pénalise les pieds qui tapent les marches
        self.rewards.feet_stumble.params["sensor_cfg"].body_names = [self.foot_link_name]

        # ---------------------------- Commande de vitesse ----------------------------
        self.commands.base_velocity.ranges.lin_vel_x = (0.0, 0.785)
        self.commands.base_velocity.ranges.lin_vel_y = (-0.20, 0.20)
        self.commands.base_velocity.ranges.ang_vel_z = (-0.20, 0.20)



        if self.__class__.__name__ == "DeeproboticsLite3StairEnvCfg":
            self.disable_zero_weight_rewards()
```

Distribution des proportions du terrains

```python
self.scene.terrain.terrain_generator.sub_terrains["random_rough"].proportion = 0.25

self.scene.terrain.terrain_generator.sub_terrains["hf_pyramid_slope"].proportion = 0.05
```

```
self.scene.terrain.terrain_generator.sub_terrains["hf_pyramid_slope_inv"].prop
ortion = 0.00
        self.scene.terrain.terrain_generator.sub_terrains["boxes"].proportion
= 0.0

self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs"].proportion
= 0.45

self.scene.terrain.terrain_generator.sub_terrains["pyramid_stairs_inv"].propor
tion = 0.25
```

IsaacLab génère un ensemble de cellules de terrain disposées en grille. Chaque cellule est assignée à un type de terrain selon les proportions définies. La somme des proportions doit faire 1.0. La majorité du terrain sont des escaliers montants comme descendant mais je conserve une partie de terrain plat et irréguliers pour conserver la marche du robot.

Section Rewards explication des récompenses attribuées

C'est la partie la plus importante du fichier. Chaque ligne modifie le comportement du robot

```
self.rewards.lin_vel_z_l2.weight = -2.0
```

Cette ligne pénalise la vitesse verticale (axe Z) du corps du robot. Sur terrain plat, on souhaite que le corps reste à hauteur constante, donc cette pénalité est forte. Sur des escaliers, les oscillations verticales parasites (rebonds, instabilités) sont dangereuses car elles peuvent faire tomber le robot. On veut que le corps monte de manière contrôlée sans rebonds, ce qui justifie de garder cette pénalité forte.

ang_vel_xy_l2 — Pénalité de vitesse angulaire (tangage et roulis)

```
self.rewards.ang_vel_xy_l2.weight = -0.5
```

Cette ligne pénalise les rotations du corps sur les axes X. La valeur est réduite par rapport à la locomotion sur terrain plat. Sur des escaliers, des légères rotations dynamiques sont inévitables et même nécessaires lors du franchissement des marches. Réduire cette pénalité donne au robot la liberté de s'incliner légèrement sur le côté pour mieux équilibrer son poids lors du passage d'une marche.

flat_orientation_l2 — Pénalité d'orientation du dos (inclinaison avant)

```
self.rewards.flat_orientation_l2.weight = -5.0
```

Cette ligne pénalise l'écart par rapport à une orientation horizontale du corps. La valeur -5.0 est plus élevée que sur terrain plat. On veut que le dos du robot reste globalement horizontal même sur les escaliers, pour éviter que le robot ne penche en avant et bascule.

feet_height — Hauteur de levée des pattes

```
self.rewards.feet_height.params["target_height"] = 0.16
```

Cette ligne récompense le robot quand ses pattes atteignent la hauteur cible des marches. La valeur de 0.16 m (16 cm) est calibrée pour correspondre à la hauteur des marches d'escalier réels. En forçant le robot à lever ses pattes à au moins 16 cm, on garantit qu'il franchisse les marches sans accrocher.

feet_air_time — Durée de phase aérienne des pattes

```
self.rewards.feet_air_time.params["threshold"] = 0.50
```

Cette ligne récompense la durée pendant laquelle une patte reste en l'air avant de toucher le sol. Le seuil de 0.50s représente le temps minimal de phase aérienne pour être récompensé. En d'autres termes, le robot est récompensé uniquement si la patte a été en l'air au moins 500 ms avant de se poser. Cette valeur force le robot à effectuer des pas plus longs et plus amples, nécessaires pour franchir des marches sans se bloquer. Des pas trop courts sur des escaliers entraînent un piétinement sur place et un blocage sur les contremarches.

feet_gait — Imposition du trot diagonal

```
self.rewards.feet_gait.weight = 1.6
```

Cette ligne récompense le fait que les pattes avant-gauche et arrière-droite se lèvent simultanément, alternant avec les pattes avant-droite et arrière-gauche. C'est l'allure naturelle des quadrupèdes pour la montée d'escaliers : elle offre une excellente stabilité car 2 pattes sont toujours en contact avec le sol, réparties en diagonale. Le poids 1.4 est une récompense forte qui incite activement l'agent à adopter cette allure spécifique.

joint_mirror — Symétrie souple des joints

```
self.rewards.joint_mirror.weight = -0.05
```

Cette ligne pénalise les asymétries entre les joints correspondants des membres gauche et droit. La valeur -0.05 est faible. On souhaite encourager une certaine symétrie pour avoir une démarche régulière, mais sans l'imposer trop strictement : sur des escaliers, des légères asymétries sont normales et souhaitables pour l'équilibrage dynamique. Un poids trop fort bloquerait les adaptations asymétriques naturelles lors du passage d'une marche.

feet_air_time_variance — Variance du rythme des pattes

```
self.rewards.feet_air_time_variance.weight = -5.0
```

Cette ligne pénalise les grandes variations de durée de phase aérienne entre les différents pas. Une variance élevée signifie que le robot fait parfois des pas très longs de façon imprévisible ce qui est un signe d'instabilité. La valeur -5.0 est assez forte pour forcer un rythme régulier. Sur des escaliers à marches uniformes, un rythme constant est optimal : le robot doit apprendre à monter les marches à cadence régulière.

base_height_l2 — Maintien de la hauteur du corps

```
self.rewards.base_height_l2.params["target_height"] = 0.35
self.rewards.base_height_l2. weight = -14.0
```

Cette ligne pénalise l'écart entre la hauteur actuelle du corps et une hauteur cible de 0.35 m. Le poids -14.0 est fort. La cible de 36 cm correspond à la hauteur de corps du robot en position de marche. Ce poids élevé empêche deux comportements indésirables : le robot qui s'accroupit pour passer sous une marche, et le robot qui se redresse trop après une marche. Maintenir une hauteur de corps constante sur des escaliers nécessite une coordination de toutes les pattes, c'est pourquoi la pénalité est si forte.

feet_slide — Glissement des pattes

```
self.rewards.feet_slide.weight = -0.2
```

Cette ligne pénalise le glissement des pattes au contact du sol : une patte posée ne doit pas translater horizontalement. Sur des escaliers, le glissement est dangereux car il peut faire déraper le robot sur le bord d'une marche. La valeur -0.2 est modérée suffisante pour décourager le glissement.

feet_stumble

```
self.rewards.feet_stumble.weight = -0.4
self.rewards.feet_stumble.params["sensor_cfg"].body_names =
[self.foot_link_name]
```

Cette ligne est conçue pour les escaliers. Il pénalise les chocs des pieds contre les contremarches. La valeur -0.4 est assez forte pour que le robot apprenne à lever suffisamment les pattes pour ne pas accrocher les marches.

Sections commandes de vitesse

```
self.commands.base_velocity.ranges.lin_vel_x = (0.0, 0.785)
self.commands.base_velocity.ranges.lin_vel_y = (-0.20, 0.20)
self.commands.base_velocity.ranges.ang_vel_z = (-0.20, 0.20)
```

Ces lignes définissent l'espace des commandes de vélocité que le robot doit apprendre à suivre. Lors de l'entraînement, IsaacLab tire aléatoirement une commande dans ces plages et la maintient pendant toute la durée d'un épisode.

- lin_vel_x = (0.0, 0.785) : la vitesse avant est toujours positive (le robot avance uniquement). Sur terrain plat les config peuvent aller jusqu'à 1.5-2.0 m/s. La valeur inférieure à 0.0 garantit que le robot n'apprend pas à reculer dans ce mode.
- lin_vel_y = (-0.20, 0.20) : faible dérive latérale autorisée. Cette plage réduite force le robot à rester centré dans l'escalier. Une commande latérale trop grande sur des escaliers étroits conduirait à des chutes sur le côté.
- ang_vel_z = (-0.20, 0.20) : faibles rotations. Le robot peut légèrement s'orienter mais ne peut pas effectuer de virages serrés sur des escaliers

6. CONCLUSION

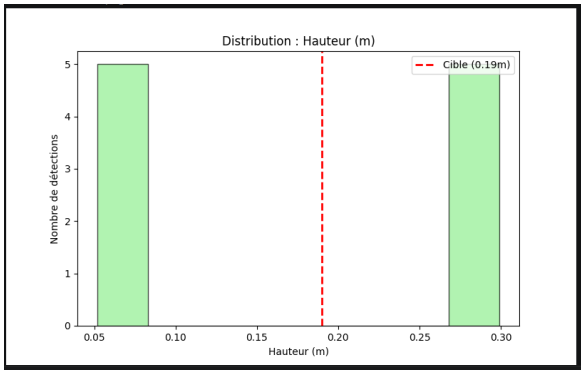
Paragraphe : Conclure sur votre Etude de Faisabilité.

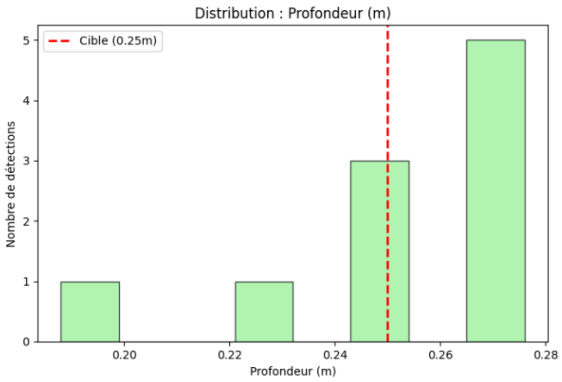
BIBLIOGRAPHIE

1. Réf 1
2. Réf 2
3. Réf 3
4. Etc.

ANNEXES

- **Realsense**
  - **Canny+Hough**






```
TERMINAL

DÉTECTION #1  |  Frame 71  |  2 marche(s) valide(s)

Marche  1  ✓
        Hauteur    :   29.9 cm
        Profondeur :   25.0 cm
        Dist caméra:   1.53 m
Marche  2  X (hors norme)
        Hauteur    :   20.9 cm
        Profondeur :    1.9 cm
        Dist caméra:   1.28 m
Marche  3  ✓
        Hauteur    :    6.6 cm
        Profondeur :   27.6 cm
        Dist caméra:   1.30 m

DÉTECTION #2  |  Frame 72  |  2 marche(s) valide(s)

Marche  1  X (hors norme)
        Hauteur    :   23.0 cm
        Profondeur :    2.5 cm
        Dist caméra:   1.50 m
Marche  2  ✓
        Hauteur    :   28.2 cm
        Profondeur :   18.8 cm
        Dist caméra:   1.47 m
Marche  3  ✓
        Hauteur    :    5.3 cm
        Profondeur :   27.0 cm
        Dist caméra:   1.29 m
```

```
DÉTECTION #1  |  Frame 75  |  2 marche(s) valide(s)

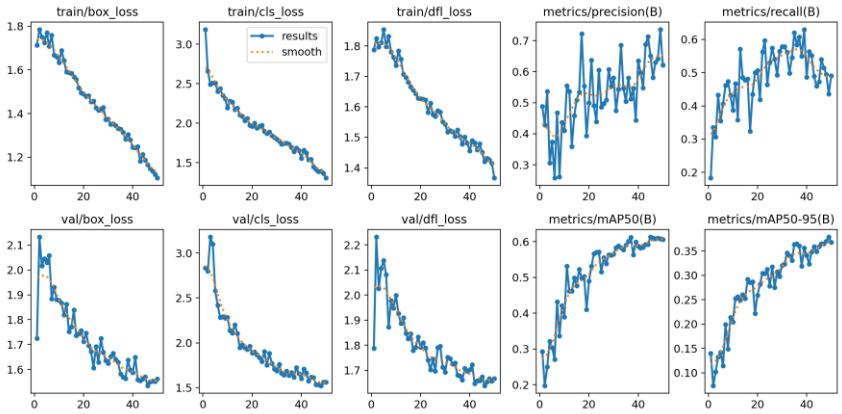
Marche  1  ✓
        Hauteur    :   29.4 cm
        Profondeur :   24.7 cm
        Dist caméra:   1.50 m
Marche  2  X (hors norme)
        Hauteur    :   21.2 cm
        Profondeur :    4.4 cm
        Dist caméra:   1.26 m
Marche  3  ✓
        Hauteur    :    5.2 cm
        Profondeur :   26.6 cm
        Dist caméra:   1.30 m

DÉTECTION #2  |  Frame 76  |  2 marche(s) valide(s)

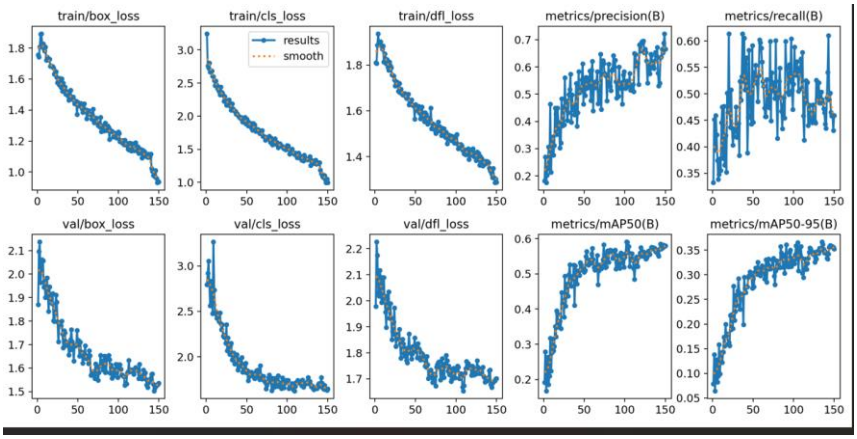
Marche  1  ✓
        Hauteur    :   29.4 cm
        Profondeur :   24.7 cm
        Dist caméra:   1.50 m
Marche  2  X (hors norme)
        Hauteur    :   21.2 cm
        Profondeur :    3.9 cm
        Dist caméra:   1.26 m
Marche  3  ✓
        Hauteur    :    6.4 cm
        Profondeur :   27.4 cm
        Dist caméra:   1.29 m
```

  - **Yolo :**

1er entrainement :

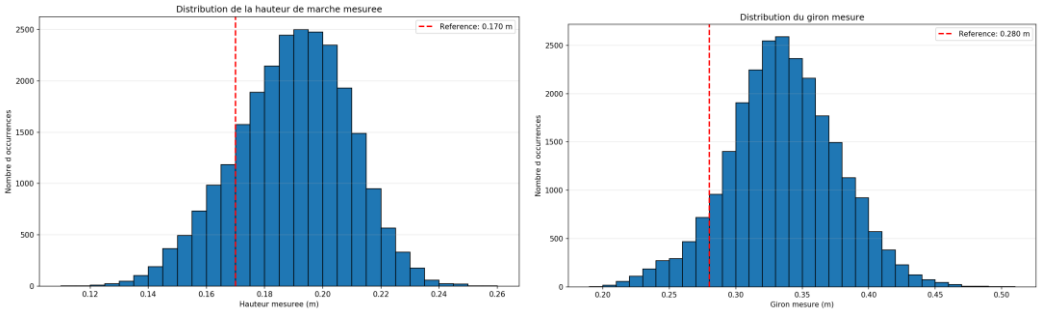


2nd entrainement :

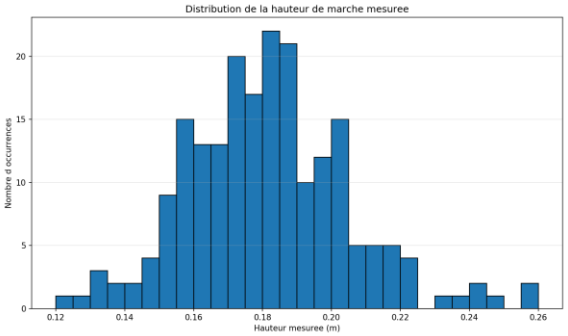


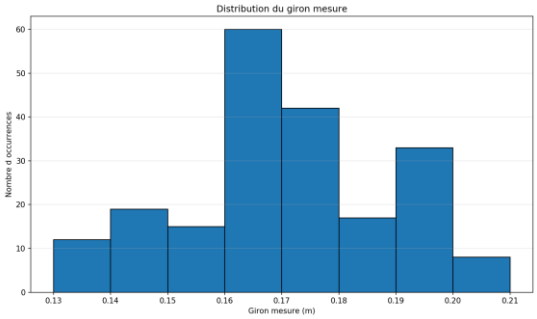
- **LIDAR**

Ransac avec petite ROI

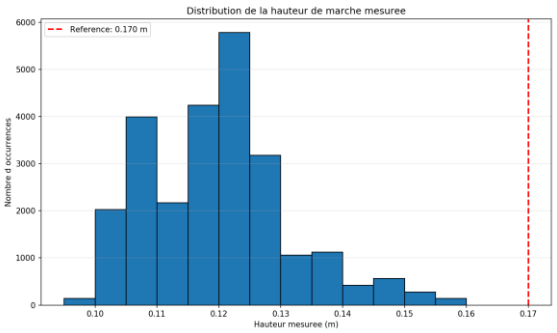


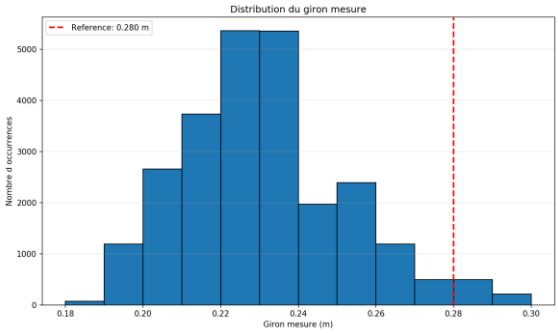
Ransac avec grande ROI



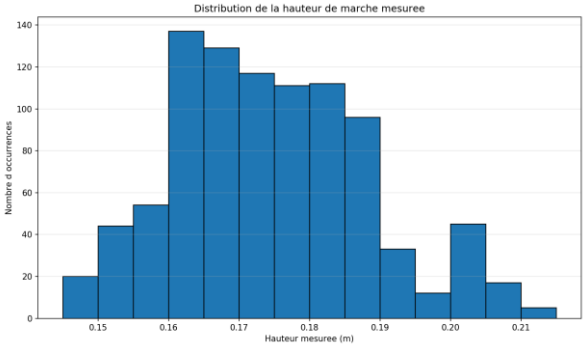


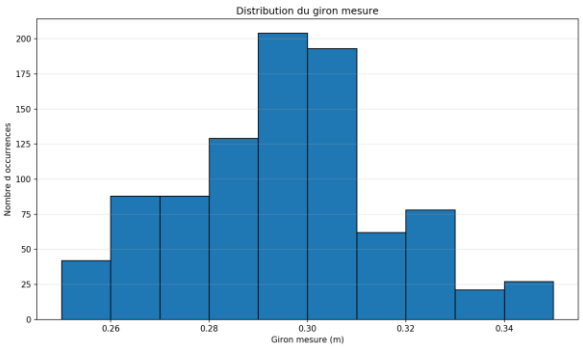
Plateau XZ avec petite ROI





Plateau XZ avec grande ROI





FIN DU DOCUMENT